

Unit 6: Client Side Scripting with JavaScript

Presented By:
Bipin Maharjan

Learning Objectives

- Understand JavaScript's role in web development and its syntax.
- Declare variables and utilize data types: numbers, strings, Booleans, null, undefined, objects, and arrays.
- Apply arithmetic, comparison, and logical operators to manipulate data.
- Implement control flow through if statements, switch statements, and ternary operators.
- Utilize loops such as while, for, and do-while for iterative processes.
- Define, call, and manage functions, including parameters, arguments, and return values.
- Create and manipulate arrays, using methods like push, pop, shift, unshift, splice, slice, and iteration techniques.
- Work with objects, access properties, employ methods, and understand object constructors and prototypes.
- Develop practical client-side scripting skills using JavaScript to enhance web interactivity and functionality.

Table of Contents

- Introduction to JavaScript
- Using JS in HTML
- JavaScript Output
- JavaScript Comments
- Variables and Data Types
- Operators and Expressions
- Control Flow and Conditionals
- Loops
- Functions
- Arrays
- Objects

Introduction to JavaScript

Introduction to JavaScript

- JavaScript is a scripting language most often used for client-side web development
- JavaScript is a high-level, interpreted programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS.
- It enables interactive web pages and is an essential part of web applications.

Key Characteristics

- Interpreted language (no compilation needed)
- Dynamically typed (variable types determined at runtime)
- Object-oriented programming support
- Event-driven programming model
- Cross-platform compatibility
- Client-side execution in web browsers

History of JavaScript

- Created by Brendan Eich at Netscape in 1995
- Originally named "Mocha", then "LiveScript", finally "JavaScript"
- Standardized as ECMAScript (ES) by ECMA International
- Current widely-used versions: ES6 (ES2015) and later

JavaScript vs Java

Despite similar names, JavaScript and Java are completely different:

Aspect	JavaScript	Java
Type	Interpreted	Compiled
Typing	Dynamically typed	Statically typed
Execution	Browser/Node.js	JVM
Syntax	C-like, flexible	C-like, strict
Use Case	Web development	Enterprise applications

Using JS in HTML

Using JS in HTML

There are three ways to include JavaScript in HTML documents:

- Inline JavaScript
- Internal JavaScript
- External JavaScript

Inline JavaScript

JavaScript code written directly in HTML element attributes.

Example:

```
<button onclick="alert('Hello World!')">Click Me</button>
```

Inline JavaScript Disadvantages

- Mixes HTML with JavaScript
- Difficult to maintain
- Cannot be cached
- Not recommended for large applications

Internal JavaScript

JavaScript code written inside `<script>` tags within the HTML document.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JS Example</title>
  <script>
    function greet() {
      alert('Hello from head section!');
    }
  </script>
</head>
```

```
<body>
  <h1>Internal JavaScript</h1>
  <button onclick="greet()">Greet</button>

  <script>
    // JavaScript in body section
    console.log('Page loaded!');
  </script>
</body>
</html>
```

Internal JavaScript: Script Placement

- In `<head>`: Loads before page content (may slow page rendering)
- In `<body>`: Loads with page content
- At end of `<body>`: Recommended - ensures HTML is loaded first

External JavaScript

JavaScript code in separate .js files, linked to HTML.

HTML file (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>External JS Example</title>
  <script src="script.js"></script>
</head>
<body>
  <h1>External JavaScript</h1>
  <button onclick="greet()">Greet</button>

  <!-- Better: Load at end of body -->
  <script src="main.js"></script>
</body>
</html>
```

JavaScript file (script.js):

```
function greet() {
    alert('Hello from external file!');
}
```

External JavaScript: Advantages

- Separation of concerns (HTML and JS separate)
- Code reusability across multiple pages
- Browser caching (faster page loads)
- Easier maintenance
- Better organization for large projects

Script Tag Attributes

```
<script src="file.js" defer></script>
```

```
<script src="file.js" async></script>
```

- defer: Script executes after HTML parsing completes
- async: Script executes as soon as downloaded (parallel)
- No attribute: Script blocks HTML parsing until executed

JavaScript Output

JavaScript Output

JavaScript provides several ways to display output:

1. `innerHTML`
2. `document.write()`
3. `window.alert()` or `alert()`
4. `console.log()`
5. `window.print()`

innerHTML

Writes content into an HTML element.

```
document.getElementById("demo").innerHTML = "Hello World!";
```

Example:

```
<p id="output"></p>
```

```
<script>
```

```
document.getElementById("output").innerHTML = "Content added by JS";
```

```
</script>
```

document.write()

Writes directly to the HTML document.

```
document.write("Hello World!");
```

Warning: Using document.write() after the page loads will overwrite the entire document. Use only for testing purposes.

Example:

```
<script>  
    document.write("<h1>Hello World!</h1>");  
</script>
```

window.alert() or alert()

Displays a popup alert box.

```
alert("Hello World!");
```

```
window.alert("Same thing!");
```

Example:

```
<script>
```

```
    alert("Welcome to our website!");
```

```
</script>
```

console.log()

Writes to the browser's developer console (F12 or Ctrl+Shift+I).

```
console.log("Debug message");
```

```
console.log("Value:", variable);
```

Other console methods:

```
console.error("Error message");    // Red error text
```

```
console.warn("Warning message");   // Yellow warning text
```

```
console.info("Info message");      // Info icon
```

```
console.table(array);              // Display as table
```

```
console.clear();                   // Clear console
```

window.print()

Opens the browser's print dialog.

```
window.print();
```


Example

```
<!DOCTYPE html>

<html>

<body>

    <h1>JavaScript Output
    Methods</h1>

    <p id="demo"></p>

    <button
onclick="showAll()">Show All
Outputs</button>
```

```
<script>
    function showAll() {
        // innerHTML

document.getElementById("demo").innerHTML =
"Changed by innerHTML";

        // alert
        alert("This is an alert box!");

        // console
        console.log("Check the console!");
        console.warn("This is a warning");
        console.error("This is an error");
    }
</script>
</body>
</html>
```

JavaScript Comments

JavaScript Comments

- Comments are used to explain code, make it more readable, and prevent execution of code during testing.
- There are two ways to comment in JavaScript:
 - Single-line comments
 - Multi-line comments

Single-line Comments

Use `//` for single-line comments.

```
// This is a single-line comment
```

```
var x = 5; // This is an inline comment
```

```
// var y = 10; // This code won't execute
```

Multi-Line Comments

Use `/* */` for multi-line comments.

```
/* This is a  
multi-line comment  
spanning multiple lines */
```

```
/*  
 * This is a formatted  
 * multi-line comment  
 * often used for documentation  
*/
```

Variable and Data Types

Variables

Variables are containers for storing data values.

Declaration Keywords:

1. `var` - Function-scoped, can be redeclared (older way)
2. `let` - Block-scoped, cannot be redeclared (ES6+)
3. `const` - Block-scoped, cannot be reassigned (ES6+)

Variable Syntax

```
var variableName = value;
```

```
let variableName = value;
```

```
const variableName = value;
```


Variable Example

Examples:

```
// Using var (function-scoped)
var name = "John";
var age = 25;
var name = "Jane"; // Allowed (redeclaration)
```

```
// Using let (block-scoped)
let city = "Kathmandu";
city = "Pokhara"; // Allowed (reassignment)
// let city = "Lalitpur"; // Error (no redeclaration)
```

```
// Using const (constant)
const PI = 3.14159;
const country = "Nepal";
// PI = 3.14; // Error (no reassignment)
```

Variable Naming Rules

- Must begin with letter, underscore (_), or dollar sign (\$)
- Subsequent characters can include digits (0-9)
- Case-sensitive (myVar and myvar are different)
- Cannot use reserved keywords (var, let, function, etc.)

Naming Conventions

// camelCase (recommended for variables/functions)

```
let firstName = "John";
```

```
let totalAmount = 100;
```

// PascalCase (for classes/constructors)

```
class UserAccount { }
```

// SCREAMING_SNAKE_CASE (for constants)

```
const MAX_SIZE = 100;
```

```
const API_KEY = "abc123";
```

Data Types

JavaScript has 8 data types:

Primitive Types (7):

1. String
2. Number
3. Boolean
4. Undefined
5. Null

Reference Type (1):

6. Object (includes arrays, functions, dates, etc.)

String

Text data enclosed in quotes.

```
let single = 'Hello';      // Single quotes
let double = "World";      // Double quotes
let backtick = `Hello World`; // Template literals
(ES6)
```

// String concatenation

```
let greeting = single + " " + double; // "Hello
World"
```

// Template literals (string interpolation)

```
let name = "John";
let message = `Hello, ${name}!`; // "Hello,
John!"
```

// String properties and methods

```
let str = "JavaScript";

console.log(str.length);    // 10

console.log(str.toUpperCase()); //
"JAVASCRIPT"

console.log(str.toLowerCase()); // "javascript"

console.log(str.charAt(0));   // "J"

console.log(str.indexOf("S")); // 4

console.log(str.slice(0, 4)); // "Java"

console.log(str.split("a"));  // ["J", "v", "Script"] 37
```

Number

Numeric values (integers and floating-point).

```
let integer = 42;  
let float = 3.14;  
let negative = -10;  
let exponential = 2.5e6;    // 2,500,000
```

// Special numeric values

```
let infinity = Infinity;  
let negInfinity = -Infinity;  
  
let notANumber = NaN;    // Result of invalid  
math operations
```

// Number methods

```
let num = 3.14159;  
  
console.log(num.toFixed(2));    // "3.14"  
console.log(num.toString());    // "3.14159"  
console.log(parseInt("42"));    // 42  
console.log(parseFloat("3.14")); // 3.14  
console.log(Number("42"));      // 42  
console.log(isNaN("hello"));    // true  
console.log(isFinite(100));     // true
```

Boolean

Logical values: true or false.

```
let isActive = true;
```

```
let isComplete = false;
```

// Boolean expressions

```
let isGreater = 10 > 5;    // true
```

```
let isEqual = 5 === "5";   // false (strict  
equality)
```

// Truthy and Falsy values

```
// Falsy: false, 0, "", null, undefined, NaN
```

```
// Truthy: everything else
```

```
if ("hello") {  
    console.log("Truthy!"); // This runs  
}
```

```
if (0) {  
    console.log("Falsy!"); // This doesn't run  
}
```

Undefined

Variable declared but not assigned a value.

```
let x;  
  
console.log(x);      // undefined  
  
console.log(typeof x); // "undefined"
```

```
function noReturn() {  
    // no return statement  
}  
  
console.log(noReturn()); // undefined
```


Null

Intentional absence of value.

```
let empty = null;  
console.log(empty);      // null  
console.log(typeof empty); // "object" (JavaScript quirk)
```

// Checking for null

```
if (empty === null) {  
    console.log("Value is null");  
}
```

Null vs Undefined:

```
let a;      // undefined (not assigned)  
let b = null; // null (intentionally empty)
```

Object

Collection of key-value pairs.

```
let person = {  
  name: "John",  
  age: 30,  
  city: "Kathmandu"  
};  
  
console.log(person.name);    // "John"  
console.log(person["age"]);  // 30
```

Type Conversion

// String conversion

```
String(123);    // "123"  
(123).toString(); // "123"
```

// Number conversion

```
Number("123");    // 123  
parseInt("123");   // 123  
parseFloat("3.14"); // 3.14  
+"123";           // 123 (unary plus)
```

// Boolean conversion

```
Boolean(1);       // true  
Boolean(0);       // false  
Boolean("");      // false  
Boolean("hello"); // true  
!!value;          // Convert to boolean
```

Operators and Expressions

What is an Operator and Expression?

- An operator is a symbol that performs an operation on one or more values.
- An expression is a combination of values, variables, and operators that produces a result.

$5 + 3$ // expression $\rightarrow$ result is 8

$x * 10$ // expression $\rightarrow$ depends on x

$a > b$ // expression $\rightarrow$ true or false

Types of Operators

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. String Operators
6. Conditional (Ternary) Operator
7. Type Operators

Arithmetic Operators

Perform mathematical calculations.

```
let a = 10, b = 3;
```

```
console.log(a + b); // 13 (Addition)
```

```
console.log(a - b); // 7 (Subtraction)
```

```
console.log(a * b); // 30 (Multiplication)
```

```
console.log(a / b); // 3.333... (Division)
```

```
console.log(a % b); // 1  
(Modulus/Remainder)
```

```
console.log(a ** b); // 1000 (Exponentiation)
```

// Increment and Decrement

```
let x = 5;
```

```
console.log(x++); // 5 (post-increment,  
returns then increments)
```

```
console.log(x); // 6
```

```
console.log(++x); // 7 (pre-increment,  
increments then returns)
```

```
let y = 5;
```

```
console.log(y--); // 5 (post-decrement)
```

```
console.log(--y); // 3 (pre-decrement)
```

Assignment Operators

Assign values to variables.

```
let x = 10;    // Basic assignment
```

```
x += 5;  // x = x + 5 -> 15
```

```
x -= 3;  // x = x - 3 -> 12
```

```
x *= 2;  // x = x * 2 -> 24
```

```
x /= 4;  // x = x / 4 -> 6
```

```
x %= 4;  // x = x % 4 -> 2
```

```
x **= 3; // x = x ** 3 -> 8
```


Comparison Operators

Compare values and return boolean.

```
let a = 5, b = "5", c = 10;
```

// Equality

```
console.log(a == b); // true (equal value, type coercion)
```

```
console.log(a === b); // false (equal value AND type)
```

// Inequality

```
console.log(a != c); // true (not equal value)
```

```
console.log(a !== b); // true (not equal value OR type)
```

Comparison Operators

// Relational

```
console.log(a > 3); // true (greater than)
```

```
console.log(a < 3); // false (less than)
```

```
console.log(a >= 5); // true (greater than or equal)
```

```
console.log(a <= 5); // true (less than or equal)
```

Important: Always prefer `===` and `!==` (strict equality) over `==` and `!=`

Logical Operators

Combine boolean expressions.

```
let a = true, b = false;
```

// AND (&&) - true if both are true

```
console.log(a && b); // false
```

```
console.log(a && a); // true
```

// OR (||) - true if at least one is true

```
console.log(a || b); // true
```

```
console.log(b || b); // false
```

// NOT (!) - inverts boolean value

```
console.log(!a);    // false
```

```
console.log(!b);    // true
```

```
console.log (!!a);   // true (double negation,  
converts to boolean)
```

String Operators

```
let firstName = "John";
```

```
let lastName = "Doe";
```

// Concatenation

```
let fullName = firstName + " " + lastName; // "John Doe"
```

// Concatenation assignment

```
let greeting = "Hello";
```

```
greeting += " World"; // "Hello World"
```

// Template literals (ES6) [Optional]

```
let message = `${firstName} ${lastName}`; // "John Doe"
```

Conditional (Ternary) Operator

Shorthand for if-else.

Syntax:

`condition ? valueIfTrue : valueIfFalse`

```
let age = 20;
```

```
let status = age >= 18 ? "Adult" : "Minor"; // "Adult"
```

Type Operator

// typeof - returns type as string

```
console.log(typeof "Hello"); // "string"
```

```
console.log(typeof 42);      // "number"
```

// instanceof - checks if object is instance of a class

```
let arr = [1, 2, 3];
```

```
console.log(arr instanceof Array); // true
```

```
console.log(arr instanceof Object); // true
```

Control Flow and Conditionals

Control Flow and Conditionals

- Control structure define the flow of program execution.
- This could allow your script to give different responses based on user input, file contents, or some other data.

Types:

- if statement
- if-else statement
- if-elseif-else statement
- Switch statement
- Ternary operator

if Statement

Execute code if condition is true.

Example

```
let age = 18;
```

```
if (age >= 18) {
```

```
    console.log("You are an adult");
```

```
}
```

if...else Statement

Execute one block if true, another if false.

Example

```
let age = 15;  
if (age >= 18) {  
    console.log("You can vote");  
} else {  
    console.log("You cannot vote yet");  
}
```

if...else if...else Statement

Check multiple conditions.

Example

```
let score = 75;  
if (score >= 90) {  
    console.log("Grade: A");  
} else if (score >= 80) {  
    console.log("Grade: B");  
} else if (score >= 70) {  
    console.log("Grade: C");  
} else if (score >= 60) {  
    console.log("Grade: D");  
} else {  
    console.log("Grade: F");  
}
```

switch Statement

Select one of many code blocks based on a value.

Example

```
let day = 3;  
let dayName;
```

```
switch (day) {  
  case 1:  
    dayName = "Sunday";  
    break;  
  case 2:  
    dayName = "Monday";  
    break;  
  case 3:  
    dayName = "Tuesday";  
    break;
```

```
  case 4:  
    dayName = "Wednesday";  
    break;  
  case 5:  
    dayName = "Thursday";  
    break;  
  case 6:  
    dayName = "Friday";  
    break;  
  case 7:  
    dayName = "Saturday";  
    break;  
  default:  
    dayName = "Invalid day";  
}  
console.log(dayName); // "Tuesday"
```

Ternary Operator

Conditional expression shorthand

Example

```
let age = 20;
```

```
let canVote = (age >= 18) ? "Yes" : "No";
```

```
console.log(canVote); // "Yes"
```

Loops

Loops

- Often when we write code, you want the same block of code to run over and over again a certain number of times. So, instead of adding several almost equal code-lines in a script, we can use loops.
- Looping statements are used to execute a block of code repeatedly based on a condition.
- Loops are useful for performing repetitive tasks, such as iterating over arrays, processing user input, generating output, etc.
- Types:
 - **for** Loop
 - **while** Loop
 - **do-while** Loop

for Loop

Best when you know how many iterations.

Syntax:

```
for (initialization; condition; increment) {  
    // code to execute  
}
```


for Loop: Examples

// Count from 1 to 5

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

// Output: 1, 2, 3, 4, 5

// Count backwards

```
for (let i = 5; i >= 1; i--) {  
    console.log(i);  
}
```

// Output: 5, 4, 3, 2, 1

// Skip by 2

```
for (let i = 0; i <= 10; i += 2) {  
    console.log(i);  
}
```

// Output: 0, 2, 4, 6, 8, 10

// Iterate over array

```
let fruits = ["Apple", "Banana", "Orange"];  
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

while Loop

Execute while condition is true. Check condition BEFORE each iteration.

Syntax:

```
while (condition) {  
    // code to execute  
}
```

while Loop: Example

```
let count = 1;  
while (count <= 5) {  
    console.log(count);  
    count++;  
}
```

// Output: 1, 2, 3, 4, 5

do...while Loop

Execute at least once, then while condition is true. Check condition AFTER each iteration.

Syntax:

```
do {  
    // code to execute  
} while (condition);
```

do...while Loop: Example

```
let count = 1;  
do {  
  console.log(count);  
  count++;  
} while (count <= 5);  
// Output: 1, 2, 3, 4, 5
```

// Executes at least once even if condition is false

```
let x = 10;  
do {  
  console.log("This runs once"); // Runs!  
} while (x < 5);
```

Jumping Statements

- Jumping statements are used to control the flow of execution by skipping or stopping iterations inside loops and conditional structures.
- Types:
 - break Statement
 - continue Statement

break - Exit loop immediately:

```
for (let i = 1; i <= 10; i++) {  
    if (i === 5) {  
        break; // Exit when i is 5  
    }  
    console.log(i);  
}  
  
// Output: 1, 2, 3, 4
```

continue - Skip current iteration:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue; // Skip when i is 3  
  }  
  console.log(i);  
}  
  
// Output: 1, 2, 4, 5
```


Functions

Introduction to Function

- A function is a block of JavaScript code that is defined once but may be executed, or invoked, any number of times
- A function can be used to return a value, construct an object, or as a mechanism to simply run code.
- JavaScript functions are defined with the function keyword
- Either function declaration or a function expression can be used

Function Structure

A function has three main parts:

- Function definition
- Function call
- Return value

Syntax

```
function functionName(parameters) {  
    // function body  
    return value;  
}
```

Function: Basic Example

```
function greet() {  
    console.log("Hello, Welcome!");  
}
```

```
greet(); // function call
```

Functions with Parameters

Parameters allow data to be passed into a function.

```
function square(num) {  
    return num * num;  
}
```

```
console.log(square(5)); // 25
```

Functions with Return Value

Why return?

- Sends result back to caller
- Stops function execution

```
function sum(a, b) {  
    return a + b;  
}
```

```
let result = sum(3, 4);  
console.log(result);
```

Function Without Return (Void Function)

```
function displayMessage(msg) {  
    console.log(msg);  
}  
  
displayMessage("JavaScript Functions");
```

Function: With Default Argument

```
function greet(name = "Guest", greeting = "Hello") {  
    return `${greeting}, ${name}!`;  
}
```

```
console.log(greet());           // "Hello, Guest!"
```

```
console.log(greet("John"));     // "Hello, John!"
```

```
console.log(greet("John", "Hi")); // "Hi, John!"
```


Arrays

Introduction to Arrays

- JavaScript arrays are used to store multiple values in a single variable
- Declaration of an Array

There are basically two ways to declare an array.

Example

```
var House = []; // method 1
```

```
var House = new array(); //method 2
```

Creating Arrays

// Array literal (preferred)

```
let fruits = ["Apple", "Banana", "Orange"];
```

// Array constructor

```
let numbers = new Array(1, 2, 3, 4, 5);
```

// Empty array

```
let empty = [];
```

// Mixed types (possible but not recommended)

```
let mixed = [1, "hello", true, null, {name: "John"}];
```

Accessing Elements

Arrays are zero-indexed (first element is at index 0).

```
let fruits = ["Apple", "Banana", "Orange"];
```

```
console.log(fruits[0]); // "Apple"
```

```
console.log(fruits[1]); // "Banana"
```

```
console.log(fruits[2]); // "Orange"
```

```
console.log(fruits[3]); // undefined
```

// Last element

```
console.log(fruits[fruits.length - 1]); // "Orange"
```

// Modify element

```
fruits[1] = "Mango";
```

```
console.log(fruits); // ["Apple", "Mango",  
"Orange"]
```

Array Properties

```
let arr = [1, 2, 3, 4, 5];
```

// Length

```
console.log(arr.length); // 5
```

// Modify length

```
arr.length = 3;
```

```
console.log(arr); // [1, 2, 3]
```

Array Methods - push() - Add to end

```
let fruits = ["Apple", "Banana"];  
fruits.push("Orange");  
console.log(fruits); // ["Apple", "Banana", "Orange"]
```

// Push multiple

```
fruits.push("Mango", "Grape");
```

pop() - Remove from end

```
let fruits = ["Apple", "Banana", "Orange"];  
let removed = fruits.pop();  
console.log(removed); // "Orange"  
console.log(fruits);  // ["Apple", "Banana"]
```

unshift() - Add to beginning

```
let fruits = ["Banana", "Orange"];  
fruits.unshift("Apple");  
console.log(fruits); // ["Apple", "Banana", "Orange"]
```


shift() - Remove from beginning

```
let fruits = ["Apple", "Banana", "Orange"];  
let removed = fruits.shift();  
console.log(removed); // "Apple"  
console.log(fruits);  // ["Banana", "Orange"]
```

splice() - Add/Remove at any position

```
let fruits = ["Apple", "Banana", "Orange", "Mango"];
```

// Remove 2 elements starting at index 1

```
let removed = fruits.splice(1, 2);  
console.log(removed); // ["Banana", "Orange"]  
console.log(fruits);  // ["Apple", "Mango"]
```

// Insert elements at index 1

```
fruits.splice(1, 0, "Grape", "Kiwi");  
console.log(fruits); // ["Apple", "Grape", "Kiwi", "Mango"]
```

// Replace 1 element at index 2

```
fruits.splice(2, 1, "Cherry");  
console.log(fruits); // ["Apple", "Grape", "Cherry", "Mango"]
```

slice() - Extract portion (doesn't modify original)

```
let fruits = ["Apple", "Banana", "Orange", "Mango"];
```

```
let citrus = fruits.slice(1, 3); // From index 1 to 3 (exclusive)
```

```
console.log(citrus); // ["Banana", "Orange"]
```

```
console.log(fruits); // ["Apple", "Banana", "Orange", "Mango"] (unchanged)
```

```
let last2 = fruits.slice(-2); // Last 2 elements
```

```
console.log(last2); // ["Orange", "Mango"]
```

concat() - Merge arrays

```
let arr1 = [1, 2, 3];
```

```
let arr2 = [4, 5, 6];
```

```
let arr3 = arr1.concat(arr2);
```

```
console.log(arr3); // [1, 2, 3, 4, 5, 6]
```

Array Methods - Iteration

forEach() - Execute function for each element

```
let fruits = ["Apple", "Banana", "Orange"];
```

```
fruits.forEach(function(fruit, index) {  
    console.log(index + ": " + fruit);  
});
```

```
// 0: Apple
```

```
// 1: Banana
```

```
// 2: Orange
```

Array Methods - sort()

```
let fruits = ["Banana", "Apple", "Orange"];  
fruits.sort();  
console.log(fruits); // ["Apple", "Banana", "Orange"]
```

```
// Numbers require compare function  
let numbers = [40, 100, 1, 5, 25];  
numbers.sort(); // [1, 100, 25, 40, 5] - Wrong! (string comparison)
```

```
numbers.sort((a, b) => a - b); // Ascending  
console.log(numbers); // [1, 5, 25, 40, 100]
```

```
numbers.sort((a, b) => b - a); // Descending  
console.log(numbers); // [100, 40, 25, 5, 1]
```

Array Methods - reverse()

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.reverse();
```

```
console.log(arr); // [5, 4, 3, 2, 1]
```

Array Methods - Converting

```
let fruits = ["Apple", "Banana", "Orange"];
```

// join() - Convert array to string

```
console.log(fruits.join());      // "Apple,Banana,Orange"  
console.log(fruits.join(" - ")); // "Apple - Banana - Orange"  
console.log(fruits.join(""));    // "AppleBananaOrange"
```

// toString() - Convert to comma-separated string

```
console.log(fruits.toString()); // "Apple,Banana,Orange"
```

// Array.from() - Create array from iterable

```
let str = "Hello";  
let chars = Array.from(str);  
console.log(chars); // ["H", "e", "l", "l", "o"]
```


Objects

Introduction to Objects

- JavaScript is an object-based language
- In JavaScript, everything is an object (except for language constructs, keywords, and operators)
- Objects plays many roles from representing data to manipulation of HTML documents via document object model (DOM), to interface with browser and more.
- An object is an unordered collection of data, including primitive types, functions and other objects.
- An object is a collection of properties, each of which has a name and a value
- JavaScript objects are dynamic. Properties can be added and deleted.

Creating Objects

Objects can be created by two ways

- Object Literal
- Constructor Function

Object Literal

Syntax: `var objectName = {};` **//Plain object**

- Creating object by defining property and values

```
let person = {  
    firstName: "John",  
    lastName: "Doe",  
    age: 30,  
    email: "john@example.com"  
};
```

Constructor Function

- Constructor function is the another way to create objects

```
function Student (pname, pcourse, pmarks) {
```

```
    this.name = pname;
```

```
    this.course = pcourse;
```

```
    this.marks = pmarks;
```

```
}
```

//The type of above object is Student name, course, marks are properties

```
student = new Student ("Mac", "Java", 65); //student is an object
```

Accessing Object (Properties)

You can access object properties in two ways:

Syntax:

`objectName.propertyName`

or

`objectName[propertyName]`

Example:

`student.name`

Accessing Properties

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 30  
};
```

// Dot notation

```
console.log(person.firstName); // "John"
```

// Bracket notation

```
console.log(person["lastName"]); // "Doe"
```

Modifying Objects

```
let person = {  
  name: "John",  
  age: 30  
};
```

// Add property

```
person.city = "Kathmandu";  
person["country"] = "Nepal";
```

// Modify property

```
person.age = 31;
```

// Delete property

```
delete person.city;
```

```
console.log(person); // {name: "John", age: 31, country: "Nepal"}
```


Object Methods

Methods are functions stored as object properties.

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  fullName: function() {  
    return this.firstName + " " + this.lastName;  
  },  
};
```

```
console.log(person.fullName()); // "John Doe"  
console.log(person.greet());   // "Hello, I'm John"
```

Iterating Over Objects

```
let person = {  
  name: "John",  
  age: 30,  
  city: "Kathmandu"  
};
```

// for...in loop

```
for (let key in person) {  
  console.log(key + ": " + person[key]);  
}
```

Object Constructors

Create multiple objects with the same structure.

```
function Person(firstName, lastName, age) {  
  this.firstName = firstName;  
  this.lastName = lastName;  
  this.age = age;  
  this.fullName = function() {  
    return this.firstName + " " + this.lastName;  
  };  
}
```

// Create instances

```
let person1 = new Person("John", "Doe", 30);  
let person2 = new Person("Jane", "Smith", 25);
```

```
console.log(person1.fullName()); // "John Doe"  
console.log(person2.fullName()); // "Jane Smith"
```

Prototypes

Objects inherit properties from their prototype.

```
function Person(name) {  
  this.name = name;  
}  
  
// Add method to prototype (shared by all instances)  
Person.prototype.greet = function() {  
  return "Hello, I'm " + this.name;  
};
```

```
let person1 = new Person("John");  
let person2 = new Person("Jane");
```

```
console.log(person1.greet()); // "Hello, I'm John"  
console.log(person2.greet()); // "Hello, I'm Jane"
```

```
// Both instances share the same method  
console.log(person1.greet === person2.greet); // true
```

Any Questions?